

Phase 4: Binary Search and Analysis — Full Structured Notes

1. Big Idea of Phase 4

Phase 4 teaches **binary search**, which is a faster way to search for a value inside an array.

In Phase 3, you learned **linear search**.

Linear search checks one element at a time:

```
A[0], A[1], A[2], A[3], ...
```

Binary search works differently.

It checks the **middle element** first. Then it removes half of the array from consideration. Then it checks the middle of the remaining part. This repeats until the key is found or the search area becomes empty.

Simple idea:

```
Binary search = check the middle, then cut the search area in half.
```

This makes binary search much faster than linear search for large arrays.

2. What You Should Already Know Before Phase 4

Before learning binary search, you should already understand:

1. What an Array ADT is.
2. The difference between `size` and `length`.
3. How to access array elements using indexes.
4. How linear search works.
5. Why search functions return an index if found and `-1` if not found.
6. How to dry-run code step by step.

The most important previous idea is:

Use `length` when working with valid elements.
Use `size` only when checking total capacity.

For binary search, this still matters.

Wrong:

```
high = arr.size - 1;
```

Correct:

```
high = arr.length - 1;
```

Because binary search should search only the valid elements, not unused capacity.

3. The Most Important Rule: The Array Must Be Sorted

Binary search works only on a **sorted array**.

Example of a sorted array:

```
[2, 4, 6, 8, 10, 12, 14]
```

The values are arranged from smallest to largest.

Binary search depends on this order.

Suppose the key is `10`.

If the middle value is `8`, then we know:

```
10 is greater than 8.
```

Because the array is sorted, all values to the left of `8` are smaller than or equal to `8`.

So `10` cannot be on the left side.

That means we can safely ignore the left half.

But if the array is not sorted, this logic fails.

Example of an unsorted array:

```
[8, 2, 14, 4, 10, 6, 12]
```

Here, checking the middle does not safely tell us which half to remove.

So the rule is:

```
Linear search works on sorted and unsorted arrays.  
Binary search works only on sorted arrays.
```

4. Why Phase 3 Improved Search Can Be Dangerous Before Binary Search

In Phase 3, you learned two improved linear search methods:

1. Transposition
2. Move-to-front

These methods move searched values closer to the front.

That can make repeated searching faster.

But they also change the order of the array.

Example:

```
[2, 3, 4, 5, 6]
```

This is sorted.

If we use move-to-front for key `5`, the array may become:

```
[5, 3, 4, 2, 6]
```

Now the array is no longer sorted.

That means binary search should not be used on it anymore.

Important warning:

Do not use transposition or move-to-front on an array that must stay sorted.

5. Binary Search Vocabulary

Binary search uses three important index variables:

Variable	Meaning
<code>low</code>	The start index of the current search area
<code>high</code>	The end index of the current search area
<code>mid</code>	The middle index of the current search area

At the beginning:

```
low = 0;  
high = arr.length - 1;
```

Then the middle index is calculated:

```
mid = (low + high) / 2;
```

In C, integer division removes the decimal part.

Example:

```
(0 + 6) / 2 = 3  
(0 + 5) / 2 = 2
```

So `mid` is always a valid whole-number index.

6. Binary Search Main Logic

Suppose we have this sorted array:

```
A = [2, 4, 6, 8, 10, 12, 14]
key = 10
```

Initial values:

```
low = 0
high = 6
mid = (0 + 6) / 2 = 3
```

So:

```
A[mid] = A[3] = 8
```

Now compare the key with the middle value.

There are three possible cases.

7. Case 1: Key Is Equal to the Middle Value

If:

```
key == A[mid]
```

Then the key is found.

Return:

```
mid
```

Example:

```
A[mid] = 10
key = 10
```

Result:

```
return mid;
```

8. Case 2: Key Is Smaller Than the Middle Value

If:

```
key < A[mid]
```

Then the key must be on the left side.

So we update:

```
high = mid - 1;
```

Why?

Because the middle value and everything to the right of it are too large.

So we keep only the left half.

9. Case 3: Key Is Greater Than the Middle Value

If:

```
key > A[mid]
```

Then the key must be on the right side.

So we update:

```
low = mid + 1;
```

Why?

Because the middle value and everything to the left of it are too small.

So we keep only the right half.

10. When Binary Search Stops

Binary search continues while:

```
low <= high
```

This means there is still a valid search area.

If this becomes false:

```
low > high
```

then the search area is empty.

That means the key is not present.

So the function returns:

```
-1
```

Simple memory:

```
low <= high means keep searching.  
low > high means stop and return -1.
```

11. Iterative Binary Search

The iterative version uses a loop.

```
int binarySearch(struct Array arr, int key) {  
    int low = 0;  
    int high = arr.length - 1;  
    int mid;  
  
    while (low <= high) {  
        mid = (low + high) / 2;  
  
        if (key == arr.A[mid]) {
```

```
        return mid;
    } else if (key < arr.A[mid]) {
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

return -1;
}
```

Explanation

```
int low = 0;
```

Start searching from the first valid index.

```
int high = arr.length - 1;
```

Search up to the last valid index.

```
while (low <= high)
```

Keep searching while the search range is not empty.

```
mid = (low + high) / 2;
```

Find the middle index.

```
if (key == arr.A[mid])
```

If the key is found, return its index.

```
else if (key < arr.A[mid])
```

If the key is smaller, search the left half.

```
else
```


If the key is greater, search the right half.

```
return -1;
```

If the loop ends, the key is missing.

12. Why Iterative Binary Search Receives the Array by Value

The iterative binary search function only reads the array.

It does not change:

1. A
2. size
3. length

So it can receive the structure by value:

```
int binarySearch(struct Array arr, int key)
```

Functions that modify the array need a pointer.

But binary search does not modify the array.

So pass-by-value is acceptable here.

13. Successful Binary Search Dry Run

Array:

```
A = [2, 4, 6, 8, 10, 12, 14]
key = 10
```

Length:

```
length = 7
```

Initial values:

```
low = 0
high = 6
```

Step	low	high	mid	A[mid]	Decision
1	0	6	3	8	10 > 8, search right
2	4	6	5	12	10 < 12, search left
3	4	4	4	10	Found

Return:

4

Why?

Because the key 10 is at index 4.

14. Unsuccessful Binary Search Dry Run

Array:

```
A = [2, 4, 6, 8, 10, 12, 14]
key = 5
```

Initial values:

```
low = 0
high = 6
```

Step	low	high	mid	A[mid]	Decision
1	0	6	3	8	5 < 8, search left
2	0	2	1	4	5 > 4, search right
3	2	2	2	6	5 < 6, search left
Stop	2	1	—	—	low > high

Now:

```
low = 2  
high = 1
```

Since:

```
low > high
```

the search area is empty.

Return:

```
-1
```

Meaning:

```
The key is not present in the array.
```

15. Binary Search With 15 Elements

A common example uses 15 sorted elements.

Why 15?

Because binary search on 15 elements forms a neat decision tree with 4 levels.

Example idea:

```
15 elements  
First check: middle element  
Then search 7 elements  
Then search 3 elements  
Then search 1 element
```

So the search range gets smaller like this:

```
15 -> 7 -> 3 -> 1
```

That is repeated halving.

16. Example Trace: Successful Search for 18

Suppose the key is 18, and it is found at index 4.

Step	low	high	mid	Logic
1	0	14	7	18 is smaller than the middle value, so move high left
2	0	6	3	18 is greater than 15, so move low right
3	4	6	5	18 is smaller than 21, so move high left
4	4	4	4	Match found

Return:

4

17. Example Trace: Successful Search for 34

Suppose the key is 34, and it is found at index 10.

Step	low	high	mid	Logic
1	0	14	7	34 is greater than the middle value, so move low right
2	8	14	11	34 is smaller than the middle value, so move high left
3	8	10	9	34 is greater than the middle value, so move low right
4	10	10	10	Match found

Return:

10

18. Example Trace: Unsuccessful Search for 25

Suppose the key is 25 and it does not exist in the array.

Step	low	high	mid	Logic
1	0	14	7	25 is smaller, so move high left
2	0	6	3	25 is greater, so move low right
3	4	6	5	25 is greater than 21, so move low right
4	6	6	6	25 is greater, so move low right
Stop	7	6	—	low > high, return -1

Result:

-1

19. Recursive Binary Search

Recursive binary search uses the same idea as iterative binary search.

The difference is that recursive binary search calls itself with a smaller search area.

Code:

```
int RBinSearch(int A[], int low, int high, int key) {
    int mid;

    if (low <= high) {
        mid = (low + high) / 2;

        if (key == A[mid]) {
            return mid;
        } else if (key < A[mid]) {
            return RBinSearch(A, low, mid - 1, key);
        } else {
            return RBinSearch(A, mid + 1, high, key);
        }
    }

    return -1;
}
```

20. Recursive Binary Search Explanation

The recursive function receives:

```
int A[]
```

The array.

```
int low
```

The start of the current search range.

```
int high
```

The end of the current search range.

```
int key
```

The value we want to find.

The condition:

```
if (low <= high)
```

means there is still a valid search range.

If the key is smaller than the middle value:

```
return RBinSearch(A, low, mid - 1, key);
```

Search the left half.

If the key is greater than the middle value:

```
return RBinSearch(A, mid + 1, high, key);
```

Search the right half.

If `low > high`, return:

-1

21. Base Case in Recursive Binary Search

A recursive function must have a stopping condition.

For recursive binary search, the base case is:

`low > high`

This means the search range is empty.

So the key is missing.

Return:

-1

Simple memory:

Recursive binary search stops when low becomes greater than high.

22. Iterative vs Recursive Binary Search

Feature	Iterative Binary Search	Recursive Binary Search
Uses loop?	Yes	No
Uses function calls?	No repeated function calls	Yes
Time complexity	$O(\log N)$	$O(\log N)$
Extra space	$O(1)$	$O(\log N)$ due to call stack
Usually preferred?	Yes	Only when recursion is required

Both versions check the same kind of middle elements.

So recursion is not faster.

The recursive version looks shorter, but it uses stack memory.

The iterative version is usually better because it avoids recursive call overhead.

23. Tail Recursion

Recursive binary search is an example of tail-style recursion because the recursive call is the final major action in each branch.

For example:

```
return RBinSearch(A, low, mid - 1, key);
```

or:

```
return RBinSearch(A, mid + 1, high, key);
```

A tail-recursive function can usually be rewritten as a loop.

That is why the iterative version is preferred.

Simple idea:

```
If recursion only calls itself at the end, a loop can usually replace it.
```

24. Coding Version Inside the Array ADT Style

The Array ADT version uses:

```
struct Array {  
    int A[20];  
    int size;  
    int length;  
};
```

Full iterative version:


```

int BinarySearch(struct Array arr, int key) {
    int low = 0;
    int high = arr.length - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (key == arr.A[mid]) {
            return mid;
        } else if (key < arr.A[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    return -1;
}

```

Important details:

1. `low` starts at `0`.
2. `high` starts at `arr.length - 1`.
3. `mid` is recalculated inside the loop.
4. The loop runs while `low <= high`.
5. The function returns `-1` only after the search range becomes empty.

25. Coding Version With Array Parameter

Recursive binary search is often written using a normal array parameter:

```

int RBinSearch(int A[], int low, int high, int key) {
    int mid;

    if (low <= high) {
        mid = (low + high) / 2;

        if (key == A[mid]) {
            return mid;
        } else if (key < A[mid]) {
            return RBinSearch(A, low, mid - 1, key);
        } else {
            return RBinSearch(A, mid + 1, high, key);
        }
    }
}

```

```
    }  
  }  
  
  return -1;  
}
```

Call example:

```
RBinSearch(arr.A, 0, arr.length - 1, key);
```

26. Dry Run: Recursive Binary Search Success

Array:

```
A = [2, 4, 5, 8, 12, 14]  
key = 5
```

Initial call:

```
RBinSearch(A, 0, 5, 5)
```

Step 1:

```
low = 0  
high = 5  
mid = (0 + 5) / 2 = 2  
A[2] = 5
```

The key is found immediately.

Return:

```
2
```

27. Dry Run: Recursive Binary Search Failure

Array:

A = [2, 4, 5, 8, 12, 14]
key = 10

Initial call:

RBinSearch(A, 0, 5, 10)

Step	low	high	mid	A[mid]	Decision
1	0	5	2	5	10 > 5, search right
2	3	5	4	12	10 < 12, search left
3	3	3	3	8	10 > 8, search right
Stop	4	3	—	—	low > high

Return:

-1

28. Why Binary Search Is Fast

Binary search is fast because it does not check every element.

It removes half of the remaining elements each time.

Example with 16 elements:

16 -> 8 -> 4 -> 2 -> 1

It takes 4 divisions by 2 to reach 1.

So:

$\log_2(16) = 4$

This is why binary search is $O(\log N)$.

Simple memory:

Linear search removes one element at a time.
Binary search removes half of the search area at a time.

29. Understanding Log in Simple Words

A logarithm answers this question:

How many times do I divide by 2 until I reach 1?

For 16:

$16 / 2 = 8$
 $8 / 2 = 4$
 $4 / 2 = 2$
 $2 / 2 = 1$

That took 4 divisions.

So:

$\log_2(16) = 4$

For 32:

$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

That takes 5 divisions.

So:

$\log_2(32) = 5$

This is much smaller than checking all 32 elements one by one.

30. Complexity of Binary Search

Let:

N = number of valid elements

Best Case

The best case happens when the key is found at the first middle position.

Only one comparison is needed.

Time complexity:

$O(1)$

Worst Case

The worst case happens when binary search must keep cutting the search range until it reaches the bottom.

Time complexity:

$O(\log N)$

Unsuccessful Case

If the key is missing, binary search must keep cutting the range until `low > high`.

Time complexity:

$O(\log N)$

Average Case

On average, the key will be found after some number of middle checks.

This also grows like:

$O(\log N)$

31. Space Complexity of Binary Search

Iterative Binary Search

The iterative version uses only a few variables:

low, high, mid

So extra space is:

$O(1)$

Recursive Binary Search

The recursive version creates function calls.

Each recursive call uses stack memory.

Because there can be about $\log N$ calls, space complexity is:

$O(\log N)$

32. Binary Search Complexity Summary

Case	Time Complexity	Reason
Best	$O(1)$	Key is at first middle position
Worst	$O(\log N)$	Search range is repeatedly halved
Unsuccessful	$O(\log N)$	Search continues until range is empty
Average	$O(\log N)$	Average number of middle checks grows logarithmically
Version	Space Complexity	Reason
Iterative	$O(1)$	Uses only variables like low, high, and mid
Recursive	$O(\log N)$	Uses call stack memory

33. Binary Search as a Decision Tree

Binary search can be understood like a tree.

For 15 elements, the first middle element is checked at the root.

Then the search can go left or right.

Each level of the tree represents one more comparison.

For 15 elements:

Tree Level	Comparisons Needed	Number of Elements at That Level
Level 1	1	1
Level 2	2	2
Level 3	3	4
Level 4	4	8

Total elements:

$$1 + 2 + 4 + 8 = 15$$

The bottom level needs the most comparisons.

This is why the worst case for 15 elements is about 4 comparisons.

34. Why Tree Height Matters

The number of comparisons in binary search depends on the height of the decision tree.

It does not directly depend on the total number of elements in the same way linear search does.

Linear search may check every element.

Binary search follows one path down the tree.

Example:

Start at the root.
Go left or right.

Go left or right again.
Continue until found or empty.

So binary search checks one element per tree level.

That is why the time is connected to tree height.

Tree height grows like:

$\log_2 N$

35. Average Case Analysis for 15 Elements

For a binary search tree with 15 elements:

Level	Comparisons Required	Number of Elements
Level 1	1	1
Level 2	2	2
Level 3	3	4
Level 4	4	8

To calculate total comparisons:

$$(1 \times 1) + (2 \times 2) + (3 \times 4) + (4 \times 8)$$

Calculate:

$$1 + 4 + 12 + 32 = 49$$

Total elements:

$$N = 15$$

Average comparisons:

$$49 / 15 \approx 3.26$$

So, on average, binary search needs around 3.26 comparisons for 15 elements.

This is close to:

$$\log_2(15)$$

So average-case binary search is:

$$O(\log N)$$

36. Internal and External Nodes

In binary search analysis, we can think about two kinds of nodes.

Node Type	Meaning
Internal node	A successful search position where an element is found
External node	An unsuccessful search position where the search fails

Internal nodes represent actual array elements.

External nodes represent gaps where the search ends unsuccessfully.

Example:

If there are N internal nodes, there are $N + 1$ external nodes.

So:

$$\text{External nodes} = N + 1$$

37. Internal Path Length and External Path Length

The curriculum uses this relationship:

$$E = I + 2N$$

Where:

Symbol	Meaning
E	External path length
I	Internal path length
N	Number of internal nodes/elements

This relationship helps analyze unsuccessful binary searches.

The key idea is simple:

Successful searches end at internal nodes.
Unsuccessful searches end at external nodes.

Both successful and unsuccessful average cases still grow like:

$$O(\log N)$$

38. Successful vs Unsuccessful Average Case

Successful Average Case

This means the key exists in the array.

The average depends on how deep the element is in the binary search decision tree.

Result:

$$O(\log N)$$

Unsuccessful Average Case

This means the key does not exist.

The search ends at an external node.

Result:

$O(\log N)$

So both successful and unsuccessful average binary search are logarithmic.

39. Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Requires sorted array?	No	Yes
Search method	Checks one by one	Checks middle and halves the range
Best case	$O(1)$	$O(1)$
Worst case	$O(N)$	$O(\log N)$
Unsuccessful case	$O(N)$	$O(\log N)$
Average case	$O(N)$	$O(\log N)$
Space	$O(1)$	$O(1)$ iterative, $O(\log N)$ recursive

Simple memory:

Linear search walks.
Binary search cuts.

40. Complete Phase 4 Program

```
#include <stdio.h>

struct Array {
    int A[20];
    int size;
    int length;
};

int BinarySearch(struct Array arr, int key) {
    int low = 0;
    int high = arr.length - 1;
    int mid;
```

```

while (low <= high) {
    mid = (low + high) / 2;

    if (key == arr.A[mid]) {
        return mid;
    } else if (key < arr.A[mid]) {
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

return -1;
}

int RBinSearch(int A[], int low, int high, int key) {
    int mid;

    if (low <= high) {
        mid = (low + high) / 2;

        if (key == A[mid]) {
            return mid;
        } else if (key < A[mid]) {
            return RBinSearch(A, low, mid - 1, key);
        } else {
            return RBinSearch(A, mid + 1, high, key);
        }
    }

    return -1;
}

int main() {
    struct Array arr = {{2, 4, 6, 8, 10, 12, 14}, 20, 7};

    int key = 10;

    int index1 = BinarySearch(arr, key);
    printf("Iterative binary search result: %d\n", index1);

    int index2 = RBinSearch(arr.A, 0, arr.length - 1, key);
    printf("Recursive binary search result: %d\n", index2);

    return 0;
}

```

Expected output:

```
Iterative binary search result: 4  
Recursive binary search result: 4
```

41. Program Walkthrough

Starting array:

```
[2, 4, 6, 8, 10, 12, 14]
```

Key:

```
10
```

Iterative Search

Initial:

```
low = 0  
high = 6  
mid = 3  
A[3] = 8
```

Since:

```
10 > 8
```

move right:

```
low = 4
```

Now:

```
low = 4  
high = 6
```

```
mid = 5
A[5] = 12
```

Since:

```
10 < 12
```

move left:

```
high = 4
```

Now:

```
low = 4
high = 4
mid = 4
A[4] = 10
```

Found.

Return:

```
4
```

Recursive Search

The recursive function checks the same middle positions.

It also returns:

```
4
```

42. Common Beginner Mistakes in Phase 4

Mistake 1: Using Binary Search on an Unsorted Array

Wrong:

```
[5, 2, 9, 1, 7]
```

Binary search cannot safely work here.

Correct:

```
[1, 2, 5, 7, 9]
```

Mistake 2: Using `size` Instead of `length`

Wrong:

```
high = arr.size - 1;
```

Correct:

```
high = arr.length - 1;
```

Reason:

```
length tells how many valid elements exist.  
size tells only total capacity.
```

Mistake 3: Updating the Wrong Boundary

If:

```
key < A[mid]
```

Correct update:

```
high = mid - 1;
```

If:

```
key > A[mid]
```

Correct update:

```
low = mid + 1;
```

Mistake 4: Returning `-1` Too Early

Wrong idea:

```
if (key != A[mid]) {  
    return -1;  
}
```

This is wrong because the key may still exist in the left or right half.

Correct idea:

```
return -1;
```

only after the loop ends or after the recursive base case fails.

Mistake 5: Forgetting the Recursive Base Case

Recursive binary search needs this condition:

```
if (low <= high)
```

If `low > high`, return:

```
-1
```

Without a proper base case, recursion may continue incorrectly.

Mistake 6: Thinking Recursive Binary Search Is Faster

Recursive binary search is not asymptotically faster.

Both iterative and recursive binary search are:

$O(\log N)$

But recursive binary search uses extra stack space.

So iterative binary search is usually preferred.

43. Phase 4 Memory Tricks

For Binary Search

Check middle, then cut half.

For Sorted Rule

No sorted order, no binary search.

For Left Side

If key is smaller, move high left.

For Right Side

If key is bigger, move low right.

For Stop Condition

low > high means not found.

For Complexity

Binary search is $O(\log N)$ because it divides by 2 again and again.

44. Phase 4 Checkpoint Questions

You understand Phase 4 when you can answer these without looking:

1. What is binary search?
2. Why must the array be sorted?
3. What does `low` mean?
4. What does `high` mean?
5. What does `mid` mean?
6. How is `mid` calculated?
7. What happens if `key == A[mid]`?
8. What happens if `key < A[mid]`?
9. What happens if `key > A[mid]`?
10. What is the iterative stopping condition?
11. What does `low > high` mean?
12. Why does binary search return `-1`?
13. Why should binary search use `length`, not `size`?
14. What is the best-case time complexity?
15. What is the worst-case time complexity?
16. What is the unsuccessful-case time complexity?
17. What is the average-case time complexity?
18. What is the space complexity of iterative binary search?
19. What is the space complexity of recursive binary search?
20. Why is recursion not faster than iteration here?
21. What is the recursive base case?
22. Why does binary search grow as $O(\log N)$?
23. How does the decision tree explain binary search?
24. What are internal nodes?
25. What are external nodes?
26. Why are unsuccessful searches also $O(\log N)$?

45. Phase 4 Final Summary

Phase 4 teaches **binary search**, a fast searching method for sorted arrays.

Binary search works by checking the middle element and removing half of the remaining search area each time.

The three important variables are:

```
low  
high  
mid
```

The core rules are:

```
If key == A[mid], return mid.  
If key < A[mid], search left.  
If key > A[mid], search right.  
If low > high, return -1.
```

Binary search is faster than linear search because it divides the problem in half each step.

Complexity summary:

```
Best case:  $O(1)$   
Worst case:  $O(\log N)$   
Average case:  $O(\log N)$   
Unsuccessful case:  $O(\log N)$ 
```

Space summary:

```
Iterative binary search:  $O(1)$   
Recursive binary search:  $O(\log N)$ 
```

Final one-line memory:

Phase 4 is about using sorted order to search faster by repeatedly checking the middle and cutting the search area in half.